

Question 1. What are the different data types in JavaScript?

Answer: JavaScript has the following data types:

1. Primitive Types:
 - Number
 - String
 - Boolean
 - Undefined
 - Null
 - Symbol
 - BigInt
2. Non-Primitive Types:
 - Object

Example:

```
let num = 42; // Number
let name = "Alice"; // String
let isActive = true; // Boolean
let nothing = null; // Null
let notDefined; // Undefined
let uniqueId = Symbol("id"); // Symbol
let bigNumber = 123n; // BigInt
let obj = { key: "value" }; // Object
```

Question 2. Explain the difference between var, let, and const.

Answer:

- var:
 - Function-scoped.
 - Can be redeclared and updated.
 - Hoisted but not block-scoped.
- let:
 - Block-scoped.
 - Can be updated but not redeclared in the same scope.
 - Hoisted but in the temporal dead zone.
- const:
 - Block-scoped.
 - Cannot be updated or redeclared.
 - The value itself is constant, but the properties of objects declared with const can be modified.

Example:

```
// var
var x = 10;
var x = 20; // Allowed
console.log(x); // 20
```

```
// let
let y = 30;
y = 40; // Allowed
console.log(y); // 40

// const
const z = 50;
z = 60; // Error: Assignment to constant variable.
```

Question 3. What is the difference between == and ===?

Answer:

- == (Abstract Equality): Compares values for equality, performing type conversion if necessary.
- === (Strict Equality): Compares values and types without performing type conversion.

Example:

```
console.log(5 == "5"); // true (type conversion happens)
console.log(5 === "5"); // false (different types)
```

Question 4. What is a closure in JavaScript?

Answer: A closure is a function that has access to its outer function's variables, even after the outer function has returned.

Example:

```
function outerFunction() {
  let outerVariable = "I am from outer scope";

  return function innerFunction() {
    console.log(outerVariable);
  };
}

const closureFunc = outerFunction();
closureFunc(); // Output: I am from outer scope
```

Question 5. What is the event loop in JavaScript?

Answer: The event loop is a mechanism in JavaScript that ensures non-blocking operations by handling asynchronous callbacks. It continuously checks the call stack and message queue for tasks to execute.

Key Components:

1. Call Stack: Where function calls are added and removed.

2. Message Queue: Holds asynchronous tasks to be executed.
3. Event Loop: Checks if the call stack is empty, then pushes tasks from the message queue to the call stack.

Example:

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("Async Task");  
}, 0);
```

```
console.log("End");
```

```
// Output:  
// Start  
// End  
// Async Task
```

Question 6. What are promises in JavaScript?

Answer: A promise is an object that represents the eventual completion (or failure) of an asynchronous operation.

States of a Promise:

1. Pending: Initial state, neither fulfilled nor rejected.
2. Fulfilled: Operation completed successfully.
3. Rejected: Operation failed.

Example:

```
const promise = new Promise((resolve, reject) => {  
  let success = true;  
  
  if (success) {  
    resolve("Promise fulfilled");  
  } else {  
    reject("Promise rejected");  
  }  
});  
  
promise  
  .then((result) => console.log(result)) // Output: Promise fulfilled  
  .catch((error) => console.error(error));
```

Question 7. What is async/await?

Answer: async and await are used to handle asynchronous code in a more readable and sequential manner.

- async: Declares a function as asynchronous and implicitly returns a promise.
- await: Pauses the execution of the async function until the promise resolves or rejects.

Example:

```
async function fetchData() {  
  try {  
    const response = await fetch("https://api.example.com/data");  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.error("Error fetching data:", error);  
  }  
}  
  
fetchData();
```

Question 8. What is the difference between null and undefined?

Answer:

- undefined: A variable that has been declared but not assigned a value.
- null: Represents the intentional absence of a value.

Example:

```
let a;  
console.log(a); // Output: undefined  
  
let b = null;  
console.log(b); // Output: null
```

Question 9. Explain the map(), filter(), and reduce() methods.

Answer:

- map(): Creates a new array by applying a function to each element of an array.
- filter(): Creates a new array with elements that pass a specified test.
- reduce(): Reduces the array to a single value by applying a function cumulatively.

Example:

```
const numbers = [1, 2, 3, 4, 5];  
  
// map
```

```
const doubled = numbers.map(num => num * 2);
console.log(doubled); // Output: [2, 4, 6, 8, 10]

// filter
const evens = numbers.filter(num => num % 2 === 0);
console.log(evens); // Output: [2, 4]

// reduce
const sum = numbers.reduce((total, num) => total + num, 0);
console.log(sum); // Output: 15
```

Question 10. What are arrow functions in JavaScript?

Answer: Arrow functions are a shorthand syntax for writing functions. They do not have their own this context and cannot be used as constructors.

Example:

```
// Regular function
function add(a, b) {
  return a + b;
}

// Arrow function
const add = (a, b) => a + b;
console.log(add(2, 3)); // Output: 5
```

Question 11: What is JavaScript, and how is it different from Java?

Answer: JavaScript is a lightweight, interpreted programming language primarily used for adding interactivity to web pages. Java, on the other hand, is a compiled, object-oriented programming language used for building platform-independent applications. JavaScript runs in the browser, while Java can run on servers and desktop applications.

Question 12: What are JavaScript data types?

Answer: JavaScript supports the following data types:

- Primitive: Number, String, Boolean, Undefined, Null, Symbol, BigInt
- Non-Primitive: Object, Array, Function

Question 13: What are promises in JavaScript?

Answer: Promises are objects that represent the eventual completion or failure of an asynchronous operation. They can be in one of three states: pending, fulfilled, or rejected.

Example:

```
const promise = new Promise((resolve, reject) => {  
  const success = true;  
  if (success) {  
    resolve("Operation successful");  
  } else {  
    reject("Operation failed");  
  }  
});
```

```
promise  
  .then((result) => console.log(result))  
  .catch((error) => console.error(error));
```

Question 14: What is event delegation?

Answer: Event delegation is a technique in which a single event listener is added to a parent element to handle events for its child elements. It takes advantage of event bubbling.

Example:

```
document.getElementById("parent").addEventListener("click", (event) => {  
  if (event.target.tagName === "BUTTON") {  
    console.log(`Button clicked: ${event.target.textContent}`);  
  }  
});
```

Question 15: Explain the this keyword in JavaScript.

Answer: The value of this depends on how a function is called:

- Global Scope: Refers to the global object (e.g., window in browsers).
- Object Method: Refers to the object the method belongs to.
- Arrow Functions: Do not have their own this; they inherit this from their surrounding context.

Question 16: What is the difference between map() and forEach()?

Answer:

- map(): Creates a new array by applying a function to each element of an array.
- forEach(): Executes a function on each element but does not return a new array.

Example:

```
const numbers = [1, 2, 3];  
const doubled = numbers.map((num) => num * 2); // [2, 4, 6]  
numbers.forEach((num) => console.log(num)); // Logs: 1, 2, 3
```

Question 17: How can you prevent default behavior in an event?

Answer: Use the `event.preventDefault()` method to prevent the default behavior of an event (e.g., stopping a form submission).

Example:

```
document.getElementById("form").addEventListener("submit", (event) => {  
    event.preventDefault();  
    console.log("Form submission prevented");  
});
```

Question 18: What is the difference between synchronous and asynchronous programming?

Answer:

- Synchronous: Tasks are executed sequentially, blocking further execution until the current task completes.
- Asynchronous: Tasks are executed without blocking, allowing the program to continue running while waiting for a task to complete.

Question 19: What are higher-order functions in JavaScript?

Answer: Higher-order functions are functions that take other functions as arguments or return them as results.

Example:

```
function higherOrderFunction(callback) {  
    callback();  
}  
  
higherOrderFunction(() => console.log("Callback executed"));
```

Question 20: Explain the concept of hoisting in JavaScript.

Answer: Hoisting is JavaScript's behavior of moving variable and function declarations to the top of their scope before execution.

Example:

```
console.log(x); // undefined  
var x = 5;
```

Question 21: What are JavaScript modules?

Answer: Modules are reusable pieces of code that can be exported from one file and imported into another using `export` and `import` keywords.

Example:

```
// module.js
export const greet = () => console.log("Hello");
```

```
// main.js
import { greet } from "./module.js";
greet();
```

Question 22: What is the purpose of the async and await keywords?

Answer: The async keyword is used to declare a function that returns a Promise. The await keyword is used inside an async function to pause execution until the Promise is resolved.

Example:

```
async function fetchData() {
  const data = await fetch("https://api.example.com/data");
  console.log(await data.json());
}
fetchData();
```

Question 23: What are JavaScript timers?

Answer: JavaScript timers allow execution of code at specified intervals using:

- `setTimeout(callback, delay)`: Executes a function after a delay.
- `setInterval(callback, delay)`: Repeatedly executes a function with a fixed time delay.

Question 24: How do you handle errors in JavaScript?

Answer: Use try...catch blocks to handle errors.

Example:

```
try {
  const result = riskyOperation();
} catch (error) {
  console.error("An error occurred:", error);
}
```

Question 25: What is the purpose of `Object.keys()` and `Object.values()`?

Answer:

- `Object.keys()`: Returns an array of a given object's own enumerable property names.
- `Object.values()`: Returns an array of a given object's own enumerable property values.

Question 26: How can you manipulate the DOM using JavaScript?

Answer: Use DOM methods like getElementById, querySelector, appendChild, and removeChild to manipulate DOM elements.

Question 27: How does the event loop work in JavaScript?

- Answer:
The event loop is a mechanism that handles the execution of multiple pieces of code, such as asynchronous callbacks. It continuously checks the call stack and the message queue. If the call stack is empty, it pushes the first task from the message queue to the stack for execution. It is responsible for managing asynchronous operations.

Question 28: What is the difference between microtasks and macrotasks in JavaScript?

- Answer:
Microtasks include operations like Promises and MutationObserver. They are executed immediately after the currently executing task completes and before macrotasks. Macrotasks include setTimeout, setInterval, and setImmediate. They are queued in the message queue and executed only after microtasks are completed.

Question 29: What is garbage collection in JavaScript, and how does it work?

- Answer:
Garbage collection is the process of reclaiming memory that is no longer being used. In JavaScript, this is done automatically using algorithms like mark-and-sweep, where objects no longer reachable from the root are marked as garbage and cleared.

Question 30: Explain currying in JavaScript with an example.

- Answer:
Currying is a technique of transforming a function with multiple arguments into a sequence of functions, each taking a single argument.

```
function multiply(a) {  
  return function (b) {  
    return a * b;  
  };  
}  
const multiplyByTwo = multiply(2);  
console.log(multiplyByTwo(3)); // Output: 6
```

Question 31: What is the difference between DOMContentLoaded and load events?

- Answer:
 - DOMContentLoaded: Fired when the initial HTML document is completely loaded and parsed, without waiting for stylesheets, images, and subframes.
 - load: Fired when the entire page, including all dependent resources (like images and stylesheets), has loaded.

Question 32: How do you prevent Cross-Site Scripting (XSS) attacks in JavaScript?

- Answer:
 - Sanitize user inputs using libraries like DOMPurify.
 - Use `textContent` instead of `innerHTML` when updating DOM elements.
 - Implement Content Security Policy (CSP) headers to restrict the sources of scripts.

Question 33: What is throttling and debouncing in JavaScript?

- Answer:
 - Throttling: Limits the execution of a function to once in a specified time frame.
 - Debouncing: Ensures a function is executed only after a specified pause time since the last call.
- Example of Debouncing:

```
function debounce(func, delay) {  
  let timeout;  
  return function (...args) {  
    clearTimeout(timeout);  
    timeout = setTimeout(() => func.apply(this, args), delay);  
  };  
}
```

Question 34: What are CORS issues, and how do you handle them?

- Answer:
Cross-Origin Resource Sharing (CORS) issues occur when a web page from one domain tries to access resources on another domain without proper authorization.
To resolve:
 - Use proper CORS headers on the server (Access-Control-Allow-Origin).
 - Enable credentials sharing if required.
 - Use JSONP or server-side proxies as alternatives.

Question 35: What is the Singleton Pattern, and how do you implement it in JavaScript?

- Answer:
The Singleton Pattern ensures a class has only one instance and provides a global point of access to it.
Example:

```
const Singleton = (function () {  
  let instance;  
  return function () {  
    if (!instance) {  
      instance = this;  
    }  
    return instance;  
  };  
})();  
const obj1 = new Singleton();  
const obj2 = new Singleton();  
console.log(obj1 === obj2); // Output: true
```

Question 36: What is the difference between fetch and XMLHttpRequest?

- Answer:
 - Fetch:
 - Returns a Promise, making it easier to handle asynchronous requests.
 - Simplified API compared to XMLHttpRequest.
 - Does not support progress events.
 - XMLHttpRequest:
 - More complex and older API.
 - Supports progress events and synchronous requests.